CELESTIAL MECHANICS × AI · 2026

# From the next token to the next theorem



Bowen Liu

# Bowen Liu 刘博文

Periodic N-body solutions, elliptic relative equilibria and linear stability.

2013–2018 **Nankai University** · Chern Institute · Math PhD

2018–2020 **University of Augsburg** · Postdoc

2020–2022 **Shanghai Jiao Tong** · Wu Wen-Tsun Asst. Prof.

2022–2024 **Huawei** · Principal Engineer

2024–now **US e-commerce** · Applied Researcher

## CELESTIAL MECHANICS RESEARCH

Periodic N-body: existence & stability

Restricted four-body elliptic equilibria

Linear instability of elliptic rhombi

Homographic Lennard–Jones solutions

> Symplectic indices · Variational methods

# After training: four limits, three system capabilities

The model predicts; state, computation and reliability live outside it.

01 · MEMORY & CONTEXT

### No persistent state, limited window

- Parameters fixed at inference; knowledge in weights
- A "conversation" resends history
- 4k → 1M, but long inputs still lose the middle

02 · COMPUTATION IS RECALL

### Recall and pattern completion

- Arithmetic comes from training patterns, not an algorithm
- It looks like computation but recalls learned mappings
- Errors compound over long tasks

03 · PROBABILISTIC OUTPUT

### Same input, different output

- Output is sampled, not a deterministic function
- Errors can sound confident and plausible
- Long runs need stopping and rollback

> Four limits, three system capabilities: **state, computation, reliability**.

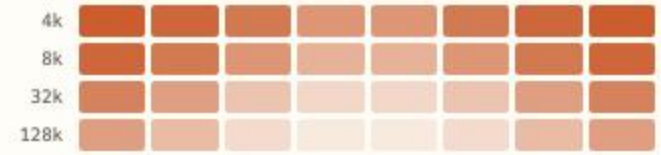
# Context and memory: a finite window, used well

The window is a budget, not a warehouse: place the right things, then decide what stays.

01 · FINITE WINDOW

## 4k → 1M, still lossy

Needle in a haystack: the middle is diluted first.




Capacity ≠ effective use.

02 · PLACEMENT

## Workbench, not warehouse

- system · rules
- history · recent turns
- retrieval · relevant docs
- tools · returned results
- current · this question

Too long→retrieve; too scattered→structure; overflow→summarize or sub-agents.

03 · RETENTION

## Signal, not logs

**Signal**
tasks · constraints · decisions · facts

**Noise**
chat · repetition · stale · raw logs

**WRITE**
decisions and facts only

**RETRIEVE**
for this task

**COMPRESS**
summarize, merge

**FORGET**
delete stale

> Capacity ≠ effective use; Context decides **what goes in**, Memory decides **what stays**.

# Tool calling: let a program do the arithmetic

tool schema

```
{
  "name": "run_python",
  "args": {"code": "str"},
  "returns": "str",
  "limits": ["sandbox", "timeout"]
}
```

one call

```
call  run_python("print(2 * 4**1.5)")
obs   "16.0"
```

```
// T = 2 × 4**1.5 = 16
```

01 DEFINE

Name, arguments, return, boundary; the description is context too.

02 CALL

The model emits a tool_call; it proposes, does not execute.

03 EXECUTE

The runtime validates arguments and runs in a sandbox; timeouts and permissions are system concerns.

04 OBSERVE

stdout / stderr return to the context; the model decides to continue or stop.

> Tool output is still **untrusted input**: validate arguments, limit permissions, set timeouts, keep logs.

> The model proposes, the program executes, **the system owns the boundary**.

# Harness: turn probabilistic output into a runnable system

The model proposes; the harness makes the proposal legal, executable, checkable, and stoppable.

A controlled loop

Propose → Validate → Execute → Observe → Verify → Decide

ENGINEERING BOUNDARY

The model only proposes token distributions and tool calls; the security and determinism of the environment are enforced by the **Harness**.

It does not add intelligence; it makes errors **visible, stoppable, reproducible**.

01 TOOL REGISTRATION & VALIDATION
Check schemas and types; block illegal proposals immediately

02 SANDBOX & PERMISSIONS
What may run; read/write isolation, constrain before execution

03 TIMEOUTS & RETRIES
Prevent infinite loops; enforce timeout limits and retry caps

04 RESULT VERIFICATION
Check correctness via independent assertions; never trust self-claims

05 ROLLBACK & STOPPING
Undo bad changes via snapshots; force-stop on error thresholds

06 LOGS & REPLAY
Record every call and response; ensure full trajectory auditability

The model provides probability; the harness provides the **boundary**.

# Put the model back in the system




| | | |
|---|---|---|
| 01 | **LLM** | A probability generator: tokens in, a distribution over the next token out |
| 02 | **Prompt** | This call's conditions: system / user / few-shot |
| 03 | **Context** | Every token visible to this call; the window is finite |
| 04 | **Memory** | State beyond the window: write / retrieve / compress / forget |
| 05 | **Agent loop** | Observe → act → observe; state lives outside |
| 06 | **Harness** | Tools, sandbox, permissions, verification, logs |

> Memory owns state, tool calls compute, harness verifies: together they form an **agent system**.

# Skill: package a procedure into a reusable capability

Not a bigger model; a load-on-demand bundle of steps, scripts, and references.

```
skill package
skill/
  SKILL.md        when · steps · checks
  scripts/        deterministic ops
  references/     domain docs
  assets/         templates
```

Outline → Generate → Screenshot → Audit → Export

01 TRIGGER
Name and description: when to use it, when not to

02 STEPS
SKILL.md: what comes first, what follows, where the checks are

03 TOOLS
scripts/: freeze deterministic operations; stop rewriting them

04 ASSETS
references/ and assets/: domain knowledge, templates, examples

A skill is loaded on demand, versioned, and reusable; context budget and tool permissions still apply.

Turn a one-off prompt into a **versioned procedure**.

# Long-running: turn the loop into a skill

The loop produces experience; the skill turns it into a procedure the next run can load.

01 · RUN

## Loop

`Observe` → `Act` ↺

State lives outside; a run can last hours or days.

02 · DISTILL

## Trajectory → steps

`Success` `Failure` `Repair` `Checkpoint`

Drop the accidental; keep what repeats.

03 · CODIFY

## Skill

```
SKILL.md
scripts/
references/
```

Load on demand; use it on the next run.

> `Loop is runtime; skill is compiled experience; memory is state; harness is the boundary.`

> Long runs do not only finish tasks; they turn practice into an **asset**.

# From the next token to the next theorem

token → probability → candidate → verification → **theorem**

Probability gives breadth

Verification gives correctness

**People set the direction**

> The next theorem still begins with the **next token**.